

BackwardCFunction#

<https://pytorch.org/docs/stable/generated/torch.autograd.function.BackwardCFunction.html>

Description:

This class is used for internal autograd work. Do not use. Apply method used when executing this Node during the backward Apply method used when executing forward mode AD during the forward Mark given tensors as modified in an in-place operation. This should be called at most once, in either the `setup_context()` or `forward()` methods, and all arguments should be inputs.

Function Signature

```
class torch.autograd.function.BackwardCFunction[source]#  
apply(*args)[source]#  
apply_jvp(*args)[source]#  
mark_dirty(*args)[source]#  
Examples::
```

Code Examples

```
>>> class Inplace(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         x_npy = x.numpy() # x_npy shares storage with x
>>>         x_npy += 1
>>>         ctx.mark_dirty(x)
>>>         return x
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, grad_output):
>>>         return grad_output
>>>
>>> a = torch.tensor(1., requires_grad=True,
>>> dtype=torch.double).clone()
>>> b = a * a
>>> Inplace.apply(a) # This would lead to wrong gradients!
>>>                 # but the engine would not know unless we
mark_dirty
>>> b.backward() # RuntimeError: one of the variables needed for
gradient
>>>             # computation has been modified by an inplace
operation
```

```
>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         sorted, idx = x.sort()
>>>         ctx.mark_non_differentiable(idx)
>>>         ctx.save_for_backward(x, idx)
>>>         return sorted, idx
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2): # still need to accept g2
>>>         x, idx = ctx.saved_tensors
>>>         grad_input = torch.zeros_like(x)
>>>         grad_input.index_add_(0, idx, g1)
>>>         return grad_input
```

```

>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x: torch.Tensor, y: torch.Tensor, z:
int):
>>>         w = x * z
>>>         out = x * y + y * z + w * y
>>>         ctx.save_for_backward(x, y, w, out)
>>>         ctx.z = z # z is not a tensor
>>>         return out
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, grad_out):
>>>         x, y, w, out = ctx.saved_tensors
>>>         z = ctx.z
>>>         gx = grad_out * (y + y * z)
>>>         gy = grad_out * (x + z + w)
>>>         gz = None
>>>         return gx, gy, gz
>>>
>>> a = torch.tensor(1., requires_grad=True, dtype=torch.double)
>>> b = torch.tensor(2., requires_grad=True, dtype=torch.double)
>>> c = 4
>>> d = Func.apply(a, b, c)

```

```

>>> class Func(torch.autograd.Function):
>>>     @staticmethod
>>>     def forward(ctx, x: torch.Tensor, y: torch.Tensor, z:
int):
>>>         ctx.save_for_backward(x, y)
>>>         ctx.save_for_forward(x, y)
>>>         ctx.z = z
>>>         return x * y * z
>>>
>>>     @staticmethod
>>>     def jvp(ctx, x_t, y_t, _):
>>>         x, y = ctx.saved_tensors
>>>         z = ctx.z
>>>         return z * (y * x_t + x * y_t)
>>>
>>>     @staticmethod
>>>     def vjp(ctx, grad_out):
>>>         x, y = ctx.saved_tensors
>>>         z = ctx.z
>>>         return z * grad_out * y, z * grad_out * x, None
>>>
>>>     a = torch.tensor(1., requires_grad=True,
dtype=torch.double)
>>>     t = torch.tensor(1., dtype=torch.double)
>>>     b = torch.tensor(2., requires_grad=True,
dtype=torch.double)
>>>     c = 4
>>>
>>>     with fwAD.dual_level():
>>>         a_dual = fwAD.make_dual(a, t)
>>>         d = Func.apply(a_dual, b, c)

```

```

>>> class SimpleFunc(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         return x.clone(), x.clone()
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2):
>>>         return g1 + g2  # No check for None necessary
>>>
>>> # We modify SimpleFunc to handle non-materialized grad
>>> outputs
>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         ctx.set_materialize_grads(False)
>>>         ctx.save_for_backward(x)
>>>         return x.clone(), x.clone()
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2):
>>>         x, = ctx.saved_tensors
>>>         grad_input = torch.zeros_like(x)
>>>         if g1 is not None:  # We must check for None now
>>>             grad_input += g1
>>>         if g2 is not None:
>>>             grad_input += g2
>>>         return grad_input
>>>
>>> a = torch.tensor(1., requires_grad=True)
>>> b, _ = Func.apply(a)  # induces g2 to be undefined

```

Detailed Documentation

Documentation

This class is used for internal autograd work. Do not use.

Apply method used when executing this Node during the backward

Apply method used when executing forward mode AD during the forward

Mark given tensors as modified in an in-place operation.

This should be called at most once, in either the `setup_context()` or `forward()` methods, and all arguments should be inputs.

Every tensor that's been modified in-place in a call to `forward()` should be given to this function, to ensure correctness of our checks. It doesn't matter whether the function is called before or after modification.

Mark outputs as non-differentiable.

This should be called at most once, in either the `setup_context()` or `forward()` methods, and all arguments should be tensor outputs.

This will mark outputs as not requiring gradients, increasing the efficiency of backward computation. You still need to accept a gradient for each output in `backward()`, but it's always going to be a zero tensor with the same shape as the shape of a corresponding output.

Save given tensors for a future call to `backward()`.

`save_for_backward` should be called at most once, in either the `setup_context()` or `forward()` methods, and only with tensors.

All tensors intended to be used in the backward pass should be saved with

`save_for_backward` (as opposed to directly on `ctx`) to prevent incorrect gradients and memory leaks, and enable the application of saved tensor hooks. See `torch.autograd.graph.saved_tensors_hooks`. See [Extending torch.autograd](#) for more details.

Note that if intermediary tensors, tensors that are neither inputs nor outputs of `forward()`, are saved for backward, your custom Function may not support double backward. Custom Functions that do not support double backward should decorate their `backward()` method with `@once_differentiable` so that performing double backward raises an error. If you'd like to support double backward, you can either recompute intermediaries based on the inputs during backward or return the intermediaries as the outputs of the custom Function. See the [double backward tutorial](#) for more details.

In `backward()`, saved tensors can be accessed through the `saved_tensors` attribute. Before returning them to the user, a check is made to ensure they weren't used in any in-place operation that modified their content.

Arguments can also be `None`. This is a no-op.

See [Extending torch.autograd](#) for more details on how to use this method.

Save given tensors for a future call to `jvp()`.

`save_for_forward` should be called at most once, in either the `setup_context()` or `forward()` methods, and all arguments should be tensors.

In `jvp()`, saved objects can be accessed through the `saved_tensors` attribute.

Arguments can also be `None`. This is a no-op.

See [Extending torch.autograd](#) for more details on how to use this method.

Set whether to materialize grad tensors. Default is True.

This should be called only from either the `setup_context()` or `forward()` methods.

If True, undefined grad tensors will be expanded to tensors full of zeros prior to calling the `backward()` and `jvp()` methods.